

Apex Retail Group: From Store Rankings to Enterprise Insight

Module Module 5 Advanced - DAX, Copilot and Advanced Report Building	Certification Alignment PL-300 Advanced Topics	Estimated Time Approx. 8 to 12 hours
--	--	--

How to Use This Brief

This is the advanced Power BI project. It builds directly on the foundations established in the S36 HR Analytics project - data modelling, Power Query, and basic DAX are assumed knowledge. This project introduces advanced DAX patterns including time intelligence and dynamic ranking, advanced report interactivity features, and Power BI Copilot integration.

The project also contains a deliberate mid-project twist in Section H. Do not read ahead to that section. When you reach it, treat it as a real stakeholder request that arrived while you were already deep into your build. Your ability to adapt existing DAX measures is part of what is being assessed.

This Project's Mindset

A senior analyst does not rebuild from scratch when requirements change. They understand their DAX well enough to modify it. That is the standard this project holds you to.

Advanced DAX is not about knowing more functions. It is about understanding filter context deeply enough to know why your measures return what they return - and how to change that without breaking everything else.

Your Scenario

You are a Senior BI Analyst at Apex Retail Group, a UK multi-brand retailer operating 60 stores across five regions under three brands: Value, Core, and Premium. Revenue has been flat across 2023 and 2024. The Board has asked the Commercial Director to identify which stores should be prioritised for investment - and which should be considered for closure.

Four stakeholders will use your dashboard. Their questions are increasingly sophisticated.

Commercial Director (primary audience):

"I need store rankings that are meaningful - not just total revenue. I want to see which stores are underperforming relative to their region and brand peers, and I want to see the trend over 24 months, not just a snapshot."

CFO:

"Show me year-on-year margin movement. I want to know whether our gross margin is compressing or holding. If it is compressing, I want to see which brand and which category is driving it."

Head of Merchandising:

"Which product categories are dragging margin down? I want to see a rolling 3-month view of margin by category so I can see seasonal patterns, not just point-in-time data."

Regional Managers (RLS restricted):

"Each Regional Manager should see only their region's stores, ranked within their region. They should see their stores' performance against regional targets, not company-wide benchmarks."

Your Datasets

You have four files. Build a star schema with one central fact table (apex_sales), one supporting fact table (apex_targets), and two dimension tables (apex_stores, apex_categories). Create a DateTable in DAX.

apex_stores.csv - dimension table, one row per store

Column	Type	What It Means
store_id	Text	Unique store identifier, e.g. STR-001. Primary key. Join key for all fact tables.
store_name	Text	Descriptive name including city, store type, and brand.
brand	Text	One of three brands: Value, Core, or Premium.
region	Text	One of five UK regions: South East, North West, Midlands, Scotland, South West.
city	Text	City where the store is located.
store_type	Text	High Street, Retail Park, or Shopping Mall.
opening_year	Number	Year the store first opened.
sq_footage	Number	Store floor area in square feet. Useful for revenue-per-sqft analysis.
enterprise_zone	True/False	Whether the store is located in a government-designated Enterprise Zone. Critical for Section H.
ez_tax_relief_pct	Number	Percentage tax relief available to Enterprise Zone stores. 0 for non-EZ stores.

apex_categories.csv - dimension table, one row per product category

Column	Type	What It Means
category_id	Text	Unique category identifier, e.g. CAT-01. Join key to fact_sales.
category_name	Text	Category name: Clothing, Electronics, Homeware, Sports, Beauty, Food and Drink, Toys, Books.
gross_margin_pct	Number	Expected gross margin percentage for this category. Used to benchmark actual margin performance.
brand_affinity	Text	Whether this category sells across all brands or is Core and Premium only.

apex_sales.csv - fact table, one row per store per category per month

Column	Type	What It Means
sale_id	Text	Unique record identifier.
store_id	Text	Join key to apex_stores.csv.
category_id	Text	Join key to apex_categories.csv.
sale_month	Date	First day of the month for this record. Format: YYYY-MM-DD. Join to DateTable.
year	Number	Year extracted from sale_month. Useful for quick year-level grouping.
month_num	Number	Month number (1-12). Useful for seasonality analysis.
units_sold	Number	Total units sold in this store-category-month combination.
transactions	Number	Number of customer transactions. Used for average basket size calculations.
revenue_gbp	Number	Total revenue in GBP for this store-category-month combination.
cost_gbp	Number	Total cost of goods sold. Enterprise Zone stores have a 8% cost loading built in.
discount_gbp	Number	Total discount value given on sales.
gross_profit_gbp	Number	Revenue minus cost. Pre-calculated for convenience - verify it matches your DAX calculation.

apex_targets.csv - fact table, one row per store per month

Column	Type	What It Means
store_id	Text	Join key to apex_stores.csv.
target_month	Date	First day of the month for this target. Format: YYYY-MM-DD.
revenue_target_gbp	Number	Monthly revenue target for this store in GBP.
footfall_target	Number	Target customer footfall for this store and month.

The star schema for this project:

apex_sales is your central fact table. apex_stores and apex_categories are dimension tables connected to it. apex_targets connects to apex_stores via store_id and to your DateTable via target_month. The DateTable connects to apex_sales via sale_month. You will have 5 tables and 4 active relationships in your model.

Important: apex_sales and apex_targets are both fact tables that connect to apex_stores. This creates an ambiguous relationship chain. Use USERELATIONSHIP in DAX when you need to bring target data into a sales-context measure.

SECTION A * Problem Framing - Define Precisely Before You Build

What This Is

Write these answers in a Word document or notes file before opening Power BI. These decisions will shape every DAX measure you write in Sections C through E.

Questions to Answer

1. Define 'underperforming store' precisely - three different ways.

The Commercial Director wants to identify underperforming stores. But underperformance can mean different things. Write three distinct, measurable definitions. For each one, state which DAX measures it would require and which stakeholder it best serves.

Example: Definition 1 - Revenue below regional average. Requires: Total Revenue measure, Regional Average Revenue measure using CALCULATE and ALLEXCEPT. Serves the Commercial Director who wants regional context, not company-wide ranking.

2. What is the right benchmarking unit - company, region, or brand?

A Value brand High Street store in Scotland should not be benchmarked against a Premium Shopping Mall in London. Write two to three sentences on how you will control for brand and store type when building comparison measures. This decision affects every RANKX and benchmark measure you write.

3. What time window does 'performance' cover?

You have 24 months of data. Write one sentence per stakeholder on what time view they care about: point-in-time, year-to-date, year-on-year comparison, or rolling trend. These will determine which time intelligence functions you use in Section D.

4. What is your working assumption about data completeness?

apex_sales contains 11,520 rows. Verify in Power Query that every store appears in every month for all 8 categories. If any store-month-category combination is missing, write how you will handle it in your measures (DIVIDE with a blank denominator, COALESCE, or IFERROR).

SECTION B * Power Query and Data Model

What This Is

Load and prepare all four files. The Power Query work here is lighter than in the S36 project because this data is cleaner. Focus your attention on the model design - particularly on how to handle two fact tables sharing a dimension, and on creating a robust DateTable that supports time intelligence.

Tasks

1. Load all four files and apply these Power Query transformations.

Table	Transformation Required
apex_stores	Convert enterprise_zone to True/False logical type. Rename query to Stores.
apex_categories	Convert gross_margin_pct to Decimal Number. Rename to Categories.
apex_sales	Convert sale_month to Date type. Convert year and month_num to Whole Number. Rename to Sales.
apex_targets	Convert target_month to Date type. Convert revenue_target_gbp to Decimal Number. Rename to Targets.

2. Create the DateTable with all required columns.

Time intelligence functions require a complete, unbroken DateTable. Create it in Home > New Table.

```
DateTable =
ADDCOLUMNS (
    CALENDAR( DATE(2023,1,1), DATE(2024,12,31) ),
    "Year",          YEAR([Date]),
    "Month Num",     MONTH([Date]),
    "Month Name",    FORMAT([Date], "MMMM"),
    "Month Short",   FORMAT([Date], "MMM"),
    "Month Year",    FORMAT([Date], "MMM YYYY"),
    "Quarter",       "Q" & QUARTER([Date]),
    "Year Quarter",  YEAR([Date]) & " Q" & QUARTER([Date]),
    "Year Month",    FORMAT([Date], "YYYY-MM"),
    "Is Current Year", IF( YEAR([Date]) = YEAR(TODAY()), TRUE(), FALSE() )
)
```

Right-click DateTable in the Fields pane > Mark as date table. Choose the Date column. Then create these relationships:

- * DateTable[Date] to Sales[sale_month] - active, Many to One, Single cross-filter
- * DateTable[Date] to Targets[target_month] - active, Many to One, Single cross-filter
- * Stores[store_id] to Sales[store_id] - active, Many to One, Single cross-filter
- * Stores[store_id] to Targets[store_id] - active, Many to One, Single cross-filter
- * Categories[category_id] to Sales[category_id] - active, Many to One, Single cross-filter

3. Create two calculated columns in the Stores table.

```
// Column 1: Store Age in Years from opening_year
Store Age =
    YEAR( TODAY() ) - Stores[opening_year]

// Column 2: Revenue Band per Square Foot
// (Used later for benchmarking by store size - created now for use in Section E)
Size Band =
    SWITCH( TRUE(),
        Stores[sq_footage] < 2000, "Small",
        Stores[sq_footage] < 4000, "Medium",
        "Large"
    )
```

SECTION C * Foundation DAX Measures - Build with VAR...RETURN from the Start

What This Is

All measures in this project must use the VAR...RETURN pattern. This is not optional. VAR...RETURN makes measures easier to debug, easier to read, and faster to calculate because intermediate results are cached. If you are not using VAR...RETURN, rewrite the measure.

```
// Example: the RIGHT way to write every measure
Gross Margin % =
VAR vRevenue    = SUM( Sales[revenue_gbp] )
VAR vCost       = SUM( Sales[cost_gbp] )
VAR vGrossProfit = vRevenue - vCost
```

```
RETURN
    IF( vRevenue = 0, BLANK(), DIVIDE( vGrossProfit, vRevenue ) )
```

Required Foundation Measures

Revenue and Cost Measures

```
Total Revenue =
VAR vRev = SUM( Sales[revenue_gbp] )
RETURN vRev

Total Cost =
VAR vCost = SUM( Sales[cost_gbp] )
RETURN vCost

Gross Profit =
VAR vRevenue = [Total Revenue]
VAR vCost     = [Total Cost]
RETURN vRevenue - vCost

Gross Margin % =
VAR vRevenue = [Total Revenue]
VAR vProfit   = [Gross Profit]
RETURN
    IF( vRevenue = 0, BLANK(), DIVIDE( vProfit, vRevenue ) )

Total Discount =
VAR vDiscount = SUM( Sales[discount_gbp] )
RETURN vDiscount

Discount Rate % =
VAR vRevenue = [Total Revenue]
VAR vDiscount = [Total Discount]
RETURN
    IF( vRevenue = 0, BLANK(), DIVIDE( vDiscount, vRevenue ) )
```

Volume and Efficiency Measures

```
Total Units =
VAR vUnits = SUM( Sales[units_sold] )
RETURN vUnits

Total Transactions =
VAR vTxn = SUM( Sales[transactions] )
RETURN vTxn

Avg Basket Size =
VAR vRevenue = [Total Revenue]
VAR vTxn      = [Total Transactions]
RETURN
    IF( vTxn = 0, BLANK(), DIVIDE( vRevenue, vTxn ) )

Revenue per Sq Ft =
VAR vRevenue = [Total Revenue]
VAR vSqFt     = SUM( Stores[sq_footage] )
RETURN
    IF( vSqFt = 0, BLANK(), DIVIDE( vRevenue, vSqFt ) )
```

Target Measures (using USERELATIONSHIP)

apex_targets connects to Stores via store_id and to DateTable via target_month. When you create a measure in the Sales context, you must use USERELATIONSHIP or CALCULATE to activate the Targets relationship explicitly.

```
Revenue Target =
VAR vTarget =
    CALCULATE(
        SUM( Targets[revenue_target_gbp] ),
        USERELATIONSHIP( DateTable[Date], Targets[target_month] )
    )
RETURN vTarget

Variance to Target =
VAR vRevenue = [Total Revenue]
VAR vTarget = [Revenue Target]
RETURN
    IF( ISBLANK( vTarget ), BLANK(), vRevenue - vTarget )

Variance to Target % =
VAR vVariance = [Variance to Target]
VAR vTarget = [Revenue Target]
RETURN
    IF( ISBLANK( vTarget ), BLANK(), DIVIDE( vVariance, vTarget ) )
```

Verify every foundation measure before moving on:

Create a matrix visual with Region on rows and all foundation measures as values. Cross-check Total Revenue for at least one region against a manual SUMIF in Excel using the raw CSV. If the numbers match, your model is correct. If they do not match, fix the model before writing any time intelligence measures.

SECTION D * Time Intelligence - The 24-Month Story

What This Is

Time intelligence functions transform your data from a static snapshot into a 24-month narrative. Every time intelligence measure requires: a properly marked DateTable, an active relationship between the DateTable and your fact table's date column, and filter context that includes a time period. If any of these three conditions are not met, the function will return incorrect results silently.

Required Time Intelligence Measures

Year-to-Date and Year-on-Year

```
Revenue YTD =
VAR vYTD = TOTALYTD( [Total Revenue], DateTable[Date] )
RETURN vYTD

// SAMEPERIODLASTYEAR: returns revenue for the same period 12 months ago
Revenue SPLY =
VAR vSPLY =
```

```

    CALCULATE(
        [Total Revenue],
        SAMEPERIODLASTYEAR( DataTable[Date] )
    )
RETURN vSPLY

Revenue vs SPLY % =
VAR vRevenue = [Total Revenue]
VAR vSPLY     = [Revenue SPLY]
RETURN
    IF( ISBLANK( vSPLY ), BLANK(), DIVIDE( vRevenue - vSPLY, vSPLY ) )

Gross Profit YTD =
VAR vYTD = TOTALYTD( [Gross Profit], DataTable[Date] )
RETURN vYTD

Gross Margin % SPLY =
VAR vMarginSPLY =
    CALCULATE(
        [Gross Margin %],
        SAMEPERIODLASTYEAR( DataTable[Date] )
    )
RETURN vMarginSPLY

```

Rolling Averages and Month-on-Month

```

// Rolling 3-Month Revenue Average
// DATESINPERIOD looks back 3 months from the last date in current filter context
Revenue Rolling 3M =
VAR vLastDate = MAX( DataTable[Date] )
VAR vResult =
    CALCULATE(
        AVERAGE( Sales[revenue_gbp] ),
        DATESINPERIOD( DataTable[Date], vLastDate, -3, MONTH )
    )
RETURN vResult

// Month-on-Month change using DATEADD
Revenue Prior Month =
VAR vPriorMonth =
    CALCULATE(
        [Total Revenue],
        DATEADD( DataTable[Date], -1, MONTH )
    )
RETURN vPriorMonth

Revenue MoM % Change =
VAR vCurrent     = [Total Revenue]
VAR vPrior       = [Revenue Prior Month]
RETURN
    IF( ISBLANK( vPrior ), BLANK(), DIVIDE( vCurrent - vPrior, vPrior ) )

// Cumulative (Running Total) Revenue
Revenue Cumulative =
VAR vMaxDate = MAX( DataTable[Date] )
VAR vResult =
    CALCULATE(
        [Total Revenue],
        DataTable[Date] <= vMaxDate,

```



```

        ALL( DataTable )
    )
RETURN vResult

```

How to test time intelligence measures:

Create a line chart with DataTable[Month Year] on the X axis and Revenue SPLY on the Y axis. The 2023 data points should all be blank (no prior year). The 2024 data points should show the corresponding 2023 monthly values. If January 2024 shows the value from January 2023, your measure is correct.

A common mistake: if Revenue SPLY shows the same value as Total Revenue, you have a circular relationship or the DataTable is not marked as a date table. Fix the date table marking first.

SECTION E * RANKX and Dynamic Benchmarking

What This Is

RANKX compares a value across a set of rows and assigns a rank. The critical skill here is controlling what 'all stores' means in your ranking context. A company-wide RANKX and a region-level RANKX require different use of ALL and ALLEXCEPT. Getting this wrong produces plausible-looking but incorrect rankings.

Required Ranking Measures

Company-Wide Rankings

```

// Rank stores by Total Revenue across the whole company
// ALL( Stores ) removes any store filter, ranking each store against all 60
Store Revenue Rank (Company) =
VAR vRevenue = [Total Revenue]
RETURN
    IF(
        ISBLANK( vRevenue ),
        BLANK(),
        RANKX(
            ALL( Stores ),
            [Total Revenue],
            vRevenue,
            DESC,
            DENSE
        )
    )

// Rank by Gross Margin % - a different definition of 'performance'
Store Margin Rank (Company) =
VAR vMargin = [Gross Margin %]
RETURN
    IF(
        ISBLANK( vMargin ),
        BLANK(),
        RANKX(
            ALL( Stores ),
            [Gross Margin %],

```

```

        vMargin,
        DESC,
        DENSE
    )
)

```

Region-Level Rankings (the harder version)

ALLEXCEPT removes all filters on a table EXCEPT the columns you specify. This is what makes region-level ranking possible - it removes the store filter but keeps the region filter, so each store is ranked only against peers in the same region.

```

// Rank each store within its own region
// ALLEXCEPT keeps the region filter, removes the store filter
Store Revenue Rank (Region) =
VAR vRevenue = [Total Revenue]
RETURN
    IF(
        ISBLANK( vRevenue ),
        BLANK(),
        RANKX(
            ALLEXCEPT( Stores, Stores[region] ),
            [Total Revenue],
            vRevenue,
            DESC,
            DENSE
        )
    )

// Store's revenue vs regional average
Region Avg Revenue =
VAR vRegionTotal =
    CALCULATE(
        [Total Revenue],
        ALLEXCEPT( Stores, Stores[region] )
    )
VAR vRegionStoreCount =
    CALCULATE(
        DISTINCTCOUNT( Stores[store_id] ),
        ALLEXCEPT( Stores, Stores[region] )
    )
RETURN
    IF( vRegionStoreCount = 0, BLANK(),
        DIVIDE( vRegionTotal, vRegionStoreCount ) )

// % above or below regional average
vs Region Avg % =
VAR vRevenue = [Total Revenue]
VAR vRegionAvg = [Region Avg Revenue]
RETURN
    IF( ISBLANK( vRegionAvg ), BLANK(),
        DIVIDE( vRevenue - vRegionAvg, vRegionAvg ) )

// Flag stores in the bottom quartile of their region
Is Bottom Quartile (Region) =
VAR vRank = [Store Revenue Rank (Region)]
VAR vStoreCount =
    CALCULATE(
        DISTINCTCOUNT( Stores[store_id] ),

```

```

        ALLEXCEPT( Stores, Stores[region] )
    )
    VAR vQuartileThreshold = CEILING( vStoreCount * 0.25, 1 )
    RETURN
        IF( vRank >= vStoreCount - vQuartileThreshold + 1, TRUE(), FALSE() )

```

Dynamic Measure Selection using Field Parameters

Field Parameters allow report users to switch between measures dynamically without changing the report. Go to Modelling > New Parameter > Fields. Add Total Revenue, Gross Profit, Gross Margin %, and Revenue MoM % Change as options. This creates a slicer that switches which measure appears on your store ranking chart.

After creating the field parameter, write a dynamic title measure:

The field parameter will generate a table called 'Performance Metric Parameter' with a column containing the selected measure name. Use SELECTEDVALUE to display the selected measure name in a chart title.

```

// Dynamic chart title that changes with the field parameter selection
Dynamic Chart Title =
VAR vMetricName =
    SELECTEDVALUE( 'Performance Metric Parameter'[Performance Metric Parameter],
        "Performance" )
VAR vRegionName =
    SELECTEDVALUE( Stores[region], "All Regions" )
RETURN
    vRegionName & " - Store Rankings by " & vMetricName

```

SECTION F * Advanced Report Features - Interactivity That Earns Its Place

What This Is

Advanced report features serve one purpose: helping users get to insight faster. Drill-through lets users move from summary to detail without cluttering the summary view. Bookmarks let you create guided narratives and toggle states. Report page tooltips give context on hover. Build each feature only if it meaningfully helps a specific stakeholder.

Task 1 - Drill-Through: Store Detail Page

Build a dedicated Store Detail page that users can navigate to by right-clicking any store in a summary visual. The page should show the full 24-month performance history for a single store.

- * Add a new report page named 'Store Detail (Drill-Through)'.
- * Add a Drill-Through filter: drag Stores[store_name] to the Drill-Through well in the Filters pane.
- * Build these visuals on the page: line chart of Revenue and Gross Profit by month, bar chart of Revenue by category, KPI card showing Revenue vs Target variance, and a text card showing the store's Enterprise Zone status and tax relief percentage.
- * Add a Back button: Insert > Buttons > Back. This returns users to the page they came from.
- * Test it: on any summary page, right-click a store name in a visual and select Drill Through > Store Detail.

Task 2 - Bookmarks: The 'Before and After' Story

You will use bookmarks in Section H to show the 'before' and 'after' views of Enterprise Zone profitability. Prepare now by learning the mechanism.

- * Create a bookmark using View > Bookmarks > Add bookmark. Name it 'Standard View'.
- * Change a slicer selection or toggle a visual's visibility. Create a second bookmark named 'EZ Adjusted View'.
- * Insert two buttons (Insert > Buttons > Blank). Set Action > Type = Bookmark for each button. Assign one to each bookmark.
- * Test that clicking the buttons toggles between the two views.

Task 3 - Report Page Tooltip

A report page tooltip shows additional context when a user hovers over a data point. Build a tooltip page for the store ranking chart.

- * Add a new report page. In the Format page panel, toggle 'Allow use as tooltip' to On. Set the canvas size to Tooltip.
- * On this page, add: a line chart showing 12-month revenue trend for the hovered store, the store's SPLY comparison, and the store's regional rank.
- * On your main store ranking visual, go to Format > Tooltips > Type = Report page. Select your tooltip page.

Task 4 - What-If Parameter

A What-If parameter creates a slicer that users can adjust to see how changes affect calculations. Create one to model the impact of a margin improvement.

- * Go to Modelling > New Parameter > Numeric Range. Name it 'Margin Improvement %'. Set Minimum = 0, Maximum = 10, Increment = 0.5.
- * Write a measure that applies the What-If adjustment to Gross Margin %:

```
Adjusted Gross Margin % =  
VAR vBaseMargin      = [Gross Margin %]  
VAR vImprovementPct = 'Margin Improvement %'[Margin Improvement % Value] / 100  
RETURN  
    IF( ISBLANK( vBaseMargin ), BLANK(), vBaseMargin + vImprovementPct )
```

- * Add the slicer to the CFO page. Label it clearly: 'Simulated Margin Improvement (percentage points)'.

SECTION G * Power BI Copilot - Three Tasks with Critical Evaluation

What This Is

Power BI Copilot can generate report pages, suggest visuals, explain measures in plain English, and write DAX. Your job is not to use everything Copilot produces. Your job is to evaluate it, correct it, and know when it is not useful.

Copilot access requirement:

Power BI Copilot requires a Microsoft Fabric or Power BI Premium per-user licence. If you do not have access, use the Power BI Q and A feature instead for Tasks 1 and 2, and screenshot the Q and A results. Task 3 (DAX explanation) can be completed using Copilot in the Power BI Service if you have access, or using ChatGPT as an alternative. Document which tool you used.

Task 1 - Generate a Summary Page with Copilot

Open the Copilot pane in Power BI Desktop or Service. Use this prompt:

"Create a summary report page for a UK retail group showing revenue by region, gross margin by brand, and month-on-month revenue trend for 2024. Use a clean layout with KPI cards at the top and charts below."

Screenshot the generated page. Then evaluate it using this framework in a written section of your submission:

Evaluation Point	What to Write
Accuracy	Did Copilot use the correct measures? Check the values in its visuals against your verified measures. Note any discrepancies.
Visual appropriateness	Did it choose the right chart type for each data story? Would you have made the same choices?
Business context	Did the generated page serve the Commercial Director's question - or is it generic? What is missing?
What you changed	List every visual you modified, added, or removed after the Copilot draft - and why.

Task 2 - Ask Copilot to Explain a Measure

Paste your 'Is Bottom Quartile (Region)' measure into Copilot or ChatGPT and use this prompt:

"Explain this Power BI DAX measure in plain English for a Regional Manager who does not know DAX. Focus on what it calculates, what it would show in a report visual, and any limitations or conditions where it might give a misleading result. Do not use technical DAX terms."

In your written submission, paste the AI explanation and annotate it: what it got right, what it missed or oversimplified, and whether a non-technical Regional Manager could act on it.

Task 3 - Ask Copilot to Write a New Measure

Ask Copilot or ChatGPT to write a DAX measure that calculates the revenue contribution of each store as a percentage of its region's total revenue. Use this prompt:

"Write a DAX measure in Power BI that calculates each store's revenue as a percentage of the total revenue for its region. The measure should return blank for stores with no revenue. Use VAR...RETURN format. The model has a Stores table with a region column, and a Sales table connected via store_id."

Test the AI-generated measure. In your written submission:

- * Paste the measure as generated
- * State whether it ran without errors
- * Verify the results: for each region, the sum of all stores' contribution percentages should equal exactly 100%. If it does not, explain why the measure is wrong.
- * Show your corrected version if changes were needed

SECTION H * THE TWIST - The Enterprise Zone Restructure

Stop. Read this message from the Finance Director before doing anything else.

"I've just reviewed your draft store rankings. There's a problem. Six of our stores are located in government-designated Enterprise Zones. These stores have been carrying an 8% cost loading that reflects higher operating and compliance costs in those locations. However, they're also entitled to a tax relief ranging from 14% to 22% of their operating costs. We've never applied that relief in our performance reporting, which means those six stores have been looking worse than they actually are for two years."

"Before you present any store closure recommendations, you need to rebuild your profitability measures to include the tax relief. The relief percentage is in the enterprise_zone and ez_tax_relief_pct columns of the Stores table. I need the before and after view in the same report."

What this means for your DAX

Every gross profit and gross margin measure you have built so far is now incorrect for Enterprise Zone stores. You need to create adjusted versions. The EZ tax relief reduces the effective cost for those stores by the relief percentage.

Tasks

1. Build the EZ-adjusted cost and profit measures.

```
// EZ-Adjusted Cost: reduces cost for Enterprise Zone stores by their relief %
// SUMX iterates row by row, applying different logic per store
EZ Adjusted Cost =
SUMX(
    Sales,
    VAR vCost      = Sales[cost_gbp]
    VAR vStoreID   = Sales[store_id]
    VAR vIsEZ      =
        RELATED( Stores[enterprise_zone] )
    VAR vReliefPct =
        RELATED( Stores[ez_tax_relief_pct] ) / 100
    RETURN
        IF( vIsEZ, vCost * ( 1 - vReliefPct ), vCost )
)
```

```

EZ Adjusted Gross Profit =
VAR vRevenue      = [Total Revenue]
VAR vAdjCost      = [EZ Adjusted Cost]
RETURN vRevenue - vAdjCost

EZ Adjusted Gross Margin % =
VAR vRevenue      = [Total Revenue]
VAR vAdjProfit    = [EZ Adjusted Gross Profit]
RETURN
    IF( vRevenue = 0, BLANK(), DIVIDE( vAdjProfit, vRevenue ) )

// Show the impact: how much does EZ relief change margin for a given store?
EZ Relief Impact (ppts) =
VAR vAdjMargin    = [EZ Adjusted Gross Margin %]
VAR vBaseMargin   = [Gross Margin %]
RETURN
    IF( ISBLANK( vAdjMargin ), BLANK(), vAdjMargin - vBaseMargin )

```

2. Rebuild the store rankings using adjusted profitability.

```

// Updated region rank using EZ-adjusted margin instead of raw revenue
Store Margin Rank - EZ Adjusted (Region) =
VAR vAdjMargin = [EZ Adjusted Gross Margin %]
RETURN
    IF(
        ISBLANK( vAdjMargin ),
        BLANK(),
        RANKX(
            ALLEXCEPT( Stores, Stores[region] ),
            [EZ Adjusted Gross Margin %],
            vAdjMargin,
            DESC,
            DENSE
        )
    )

// Updated bottom quartile flag using EZ-adjusted ranking
Is Bottom Quartile - EZ Adjusted =
VAR vRank      = [Store Margin Rank - EZ Adjusted (Region)]
VAR vStoreCount =
    CALCULATE(
        DISTINCTCOUNT( Stores[store_id] ),
        ALLEXCEPT( Stores, Stores[region] )
    )
VAR vThreshold = CEILING( vStoreCount * 0.25, 1 )
RETURN
    IF( vRank >= vStoreCount - vThreshold + 1, TRUE(), FALSE() )

```

3. Build the Before and After comparison view using bookmarks.

Use the bookmarks you created in Section F to present both views in the same report page. The Commercial Director needs to see exactly which stores changed rank and by how much.

- * Create a table visual showing: store name, original margin rank, EZ-adjusted margin rank, rank change, and a conditional formatting flag for EZ stores.
- * Apply conditional formatting to the rank change column: green for stores that improved ranking, red for stores that did not change.

- * Create two bookmarks: 'Before EZ Adjustment' (showing original margin rank) and 'After EZ Adjustment' (showing EZ-adjusted rank).
- * Add toggle buttons linking to each bookmark. Label them clearly.

4. Write the required Impact Analysis section (100 words).

In your written submission, write approximately 100 words addressing all of the following:

- * How many of the six EZ stores changed their regional ranking as a result of the adjustment?
- * Were any stores that appeared in the bottom quartile before the adjustment now above the bottom quartile threshold?
- * What would have been the consequence of recommending store closures based on the unadjusted data?
- * What governance process should be in place to ensure cost adjustments like this are applied systematically in future?

SECTION I * Report Pages and Final Assembly

What This Is

Assemble your final dashboard. You have five required report pages. Design decisions are yours to make. Required visuals are specified per page. Every page must have a title that states its audience and purpose, and at least one insight annotation added via a text box or callout.

Page	Audience	Required Visuals
1 - Executive Summary	Commercial Director and Board	KPI cards: Total Revenue, YoY change, Gross Margin %, Margin SPLY. Line chart: Revenue and Revenue SPLY by month. Bar chart: Revenue by region with YoY comparison. Field parameter slicer for dynamic measure switching.
2 - Store Performance and Rankings	Commercial Director	RANKX table: stores ranked by field parameter selection, with regional rank, company rank, and Bottom Quartile flag. Map visual if coordinates available. Brand and region slicers. Drill-through enabled to Store Detail page.
3 - Margin and Category Analysis	CFO and Head of Merchandising	Line chart: rolling 3-month gross margin by category. Bar chart: gross margin % by category vs benchmark from apex_categories. What-If parameter slicer for margin simulation. Revenue vs Target variance by store.
4 - Enterprise Zone Before and After	Commercial Director and Finance Director	Bookmark toggle buttons (Before / After). Table showing rank changes. EZ stores highlighted with conditional formatting. EZ Relief Impact measure shown per EZ store. Clear narrative annotation explaining what changed.
5 - Store Detail (Drill-Through)	All stakeholders - accessed via drill-through only	24-month revenue and profit trend. Revenue by category. Revenue vs Target KPI. Store metadata: brand, region, type, size band, EZ status. Back navigation button.

RLS - Regional Managers

Create five RLS roles - one per region. Apply the same DAX pattern from the S36 project. Test each role using View As. Add a sixth role named Commercial Director with no filter.

```
// Role: South East Manager
// Filter applied to Stores table:
[region] = "South East"

// Repeat for North West, Midlands, Scotland, South West
// Commercial Director role: no filter applied
```

SECTION J * Deliverables and Submission

What to Submit

Item	Format	What It Must Contain
1 - Power BI Desktop file	.pbix	All five report pages. All DAX measures including EZ-adjusted versions. Field parameter. What-If parameter. Bookmarks for Before/After. Drill-through on Page 2. Report page tooltip. Six RLS roles.
2 - Screenshot pack	.pdf	One screenshot per report page. Model view showing all 5 tables and 4 relationships. Manage Roles screen. View As test for two regional roles. Copilot outputs from all three tasks.
3 - Written submission	.docx or .pdf	Section A framing answers. Copilot evaluation for all 3 tasks. EZ Impact Analysis (100 words). Dashboard Design Decisions per page. One insight summary per stakeholder using What/So What/Now What.
4 - Peer presentation	Live	5 minutes. Must include the Before/After bookmark toggle live. Must be able to answer: 'Why did you use ALLEXCEPT instead of ALL in your region rank measure?'

Section	Must Be Present
A - Framing	Three definitions of underperforming store. Time window per stakeholder. Benchmarking unit decision. Assumption about data completeness.
B - Model	Five tables. DateTable marked. Four relationships. Star schema correct in Model view. Two calculated columns in Stores.
C - Foundation	All foundation measures in Measures table. VAR...RETURN used throughout. Foundation measures verified against raw CSV for at least one region.
D - Time Intel	Revenue YTD, SPLY, Rolling 3M, MoM % and Cumulative Revenue all present and tested. Time intelligence test chart screenshot included.
E - Ranking	Company rank, Region rank (ALLEXCEPT), Regional average, vs Region Avg %, Bottom Quartile flag. Field parameter created. Dynamic title measure working.
F - Interactivity	Drill-through on Store Detail page. Bookmarks for Before/After (used in Section H). Report page tooltip on store ranking visual. What-If parameter with adjusted margin measure.
G - Copilot	Task 1: generated page screenshot, four-point evaluation written. Task 2: AI explanation pasted and annotated. Task 3: AI-generated measure tested, corrected if wrong, verified to sum to 100%.

H - Twist	SUMX-based EZ-adjusted cost measure. EZ-adjusted rank and bottom quartile measures. Bookmark toggle with conditional formatting. 100-word impact analysis.
I - Pages	All five pages with required visuals. Every page has a title and at least one insight annotation. RLS tested for two regional roles.

Portfolio Entry

"Built an advanced Power BI solution for a simulated UK retail group: 5-table star schema, 25-plus DAX measures including time intelligence (YTD, SPLY, rolling averages), dynamic RANKX at company and regional level using ALLEXCEPT, SUMX-based cost restatement for tax zone adjustments, field parameters, drill-through, bookmarks, and Power BI Copilot integration with critical evaluation. Delivered a Before/After scenario toggle that changed store closure recommendations for 6 stores."